

5. 합성곱 신경망 I

AI/데이터

2025/09/20 20:57

<http://blog.naver.com/sirutio03/224015560254>

5.1 합성곱 신경망

이미지 전체를 한 번에 계산하지 않고, 국소적 부분을 계산해 시간과 자원을 절약.(세밀한 부분까지 분석)

- 합성곱층의 필요성

그림을 펼쳐서 각 픽셀에 가중치를 곱하여 은닉층으로 전달하면 데이터의 공간적 구조를 무시하게 됨.

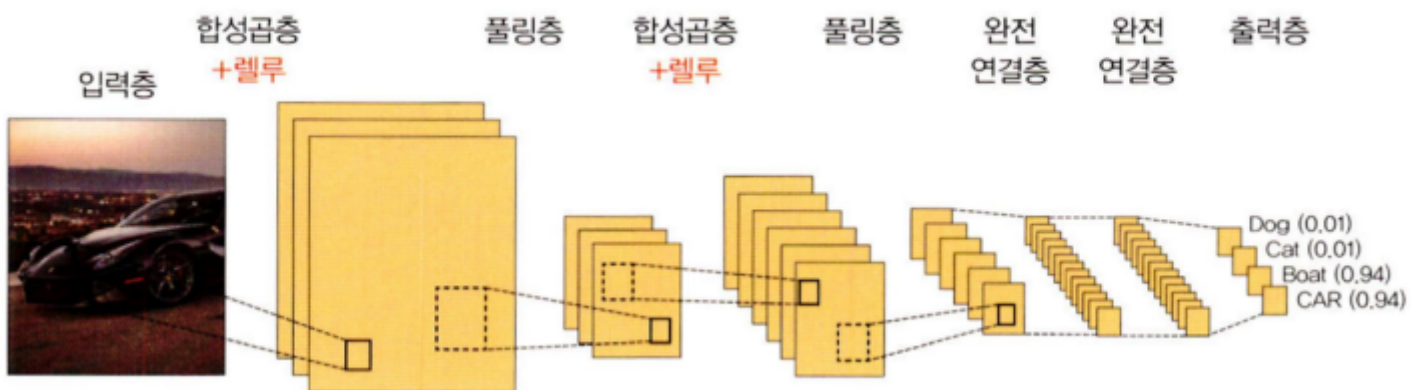
이를 방지하려 도입

- 합성곱 신경망 구조

합성곱, 풀링층을 거치며 이미지 주요 특성 벡터 추출, 완전연결층 거치면서 1차원 벡터로 변환.

출력층에서 소프트맥스 함수를 사용해 최종 결과 출력.

▼ 그림 5-2 합성곱 신경망 구조



1. 입력층

: 입력 이미지 데이터가 최초로 거치는 계층, 이미지는 height/width/channel 값을 갖추는 3차원 데이터로, 채널값은 그 레이스케일은 1, 컬러면 3을 가짐.

2. 합성곱층

: 입력 데이터에서 특성 추출하여 특성 맵 완성

1단계. 입력 이미지에 3x3 필터(커널) 적용

2단계. 필터가 스트라이드(1)만큼 이동

3단계 ... 이를 반복하며 마지막까지 이동하여 feature map 채움

+ 컬러 이미지 합성곱은 RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해 줌.

▼ 그림 5-4 입력 이미지에 3×3 필터 적용



▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동

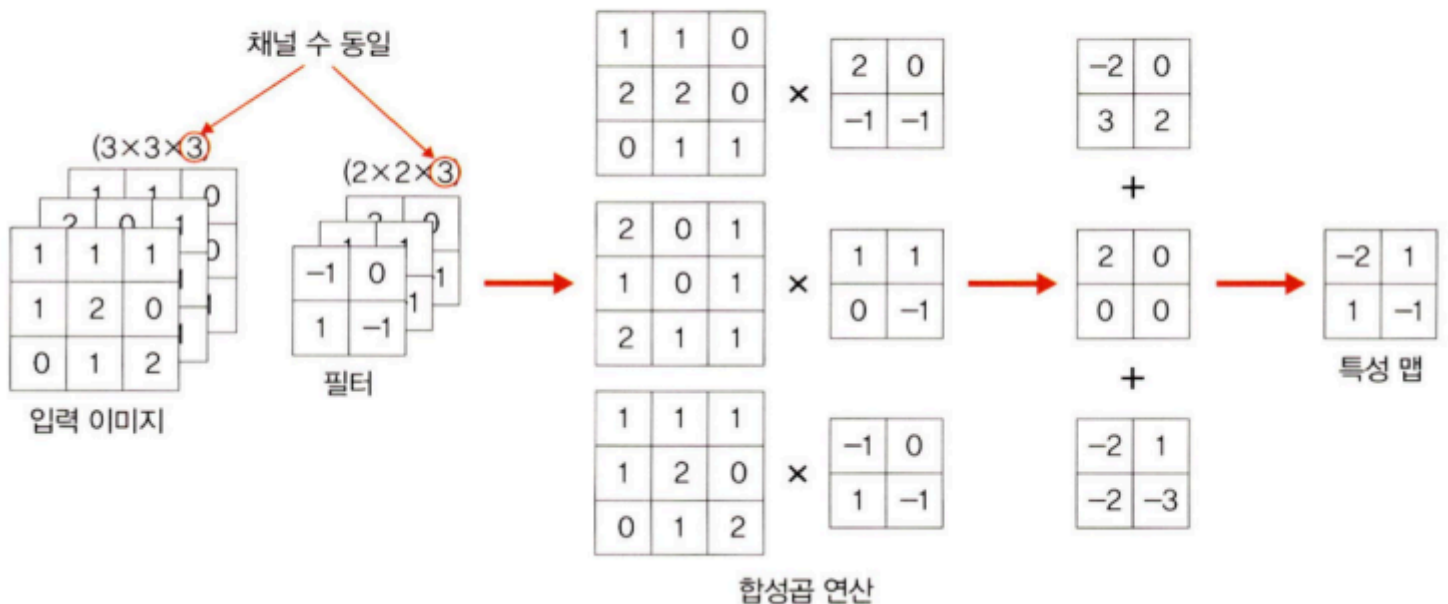


▼ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동



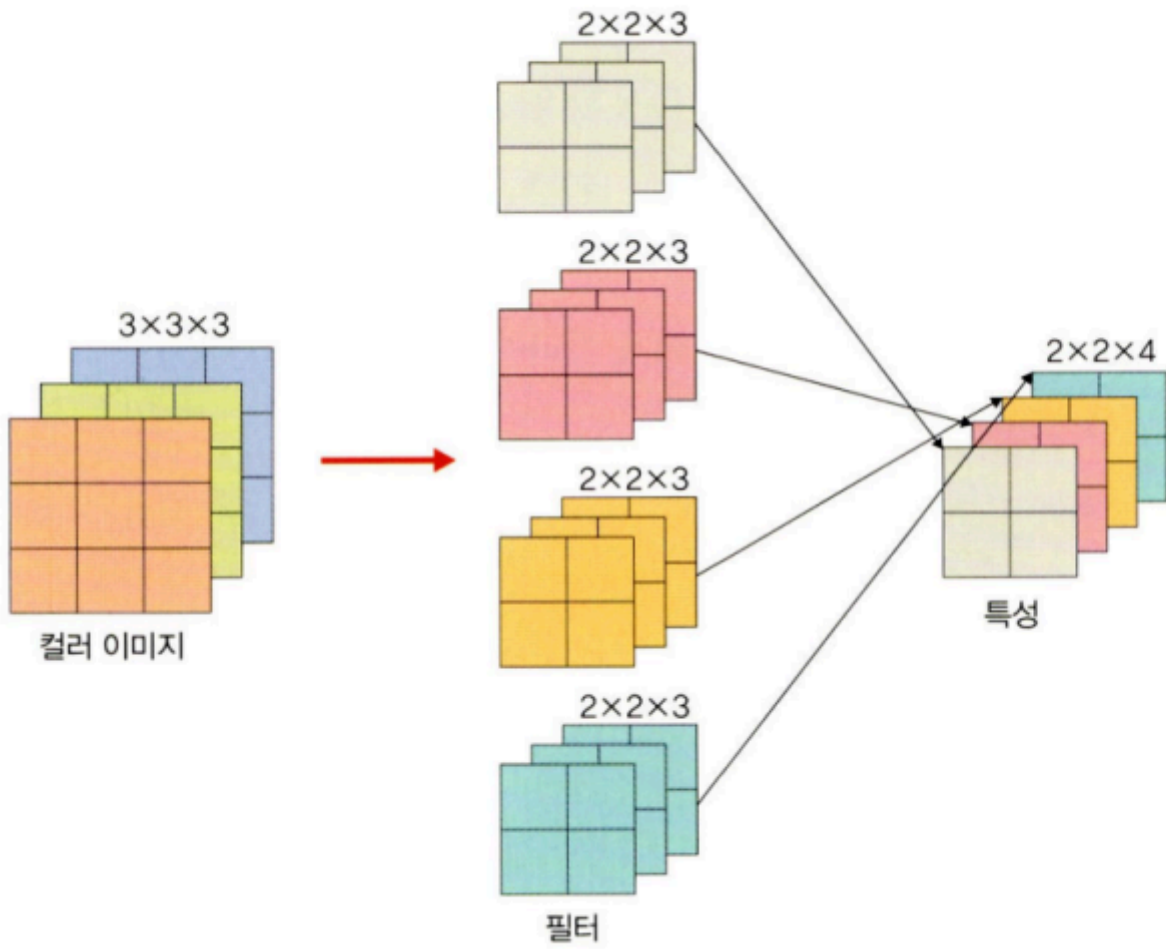
2-1. 그레이스케일: 합성곱층에서 필터가 이동하며 특성 맵을 완성하는 과정

▼ 그림 5-10 컬러 이미지 합성곱



2-2. 컬러 이미지: 그레이스케일과 동일하나 RGB마다 가중치가 다름

♥ 그림 5-11 필터가 2 이상인 합성곱



- 입력 데이터: $W_1 \times H_1 \times D_1$ (W_1 : 가로, $\times H_1$: 세로, $\times D_1$: 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

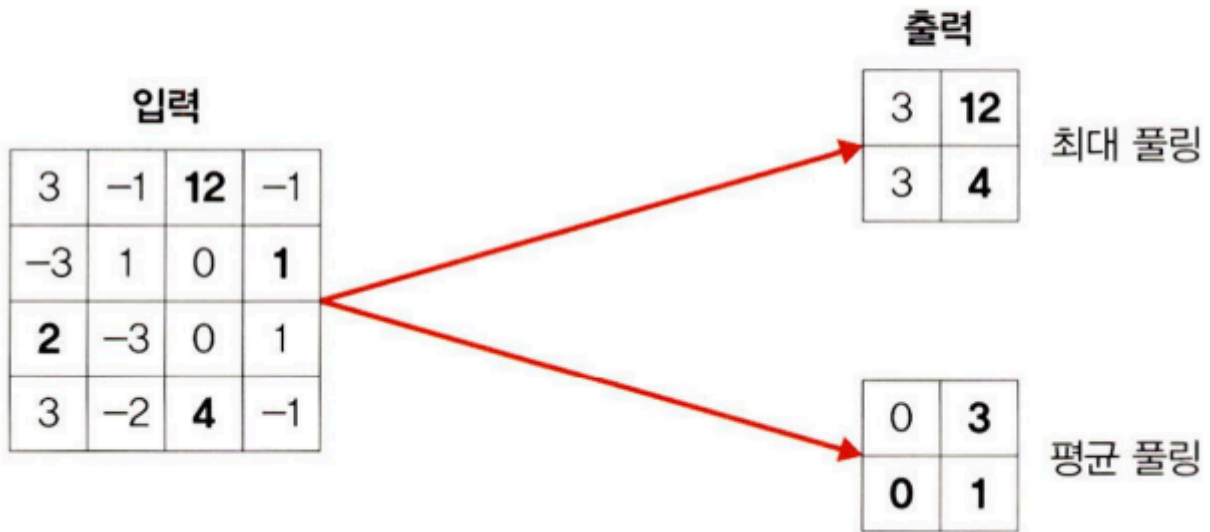
필터가 여러 개인 경우 특성 맵의 채널이 늘어남

3. 풀링층

: 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출

1. 최대 풀링: 대상 영역에서 최대값을 추출
2. 평균 풀링: 대상 영역에서 평균을 반환(각 커널 값을 평균화 시켜 중요한 가중치를 갖는 값 희미해짐)

▼ 그림 5-17 최대 풀링과 평균 풀링 비교



- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

최대 풀링과 평균 풀링

4. 완전연결층

3차원 벡터에서 1차원 벡터로 펼쳐짐

5. 출력층

소프트맥스 활성화 함수가 사용되어 이미지가 각 레이블에 속할 확률값이 출력됨

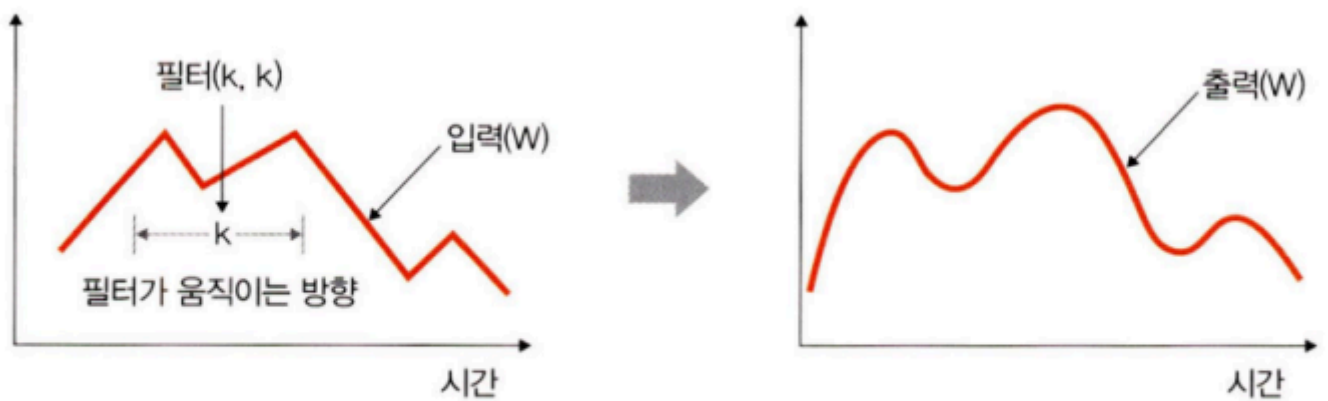
- 1D 합성곱

: 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱, 출력 형태가 1D형태의 배열

- 2D 합성곱

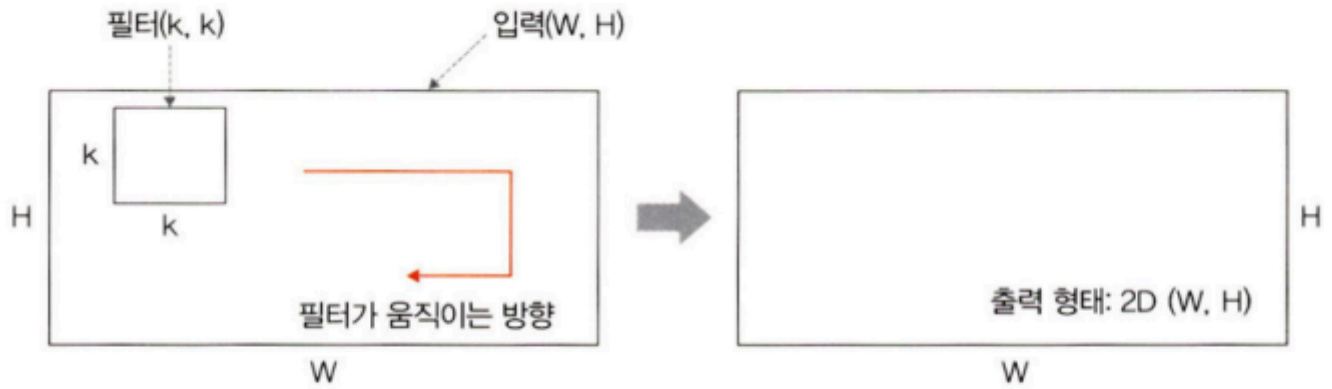
: 필터가 방향 두 개로 움직여 2D 행렬 형태로 출력하게 됨

▼ 그림 5-19 1D 합성곱



- 입력: W 너비(Width)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width)

♥ 그림 5-20 2D 합성곱



- 입력: W 너비(Width), H 높이(Height)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width), H 높이(Height)

1D 합성곱, 2D 합성곱

- 3D 합성곱

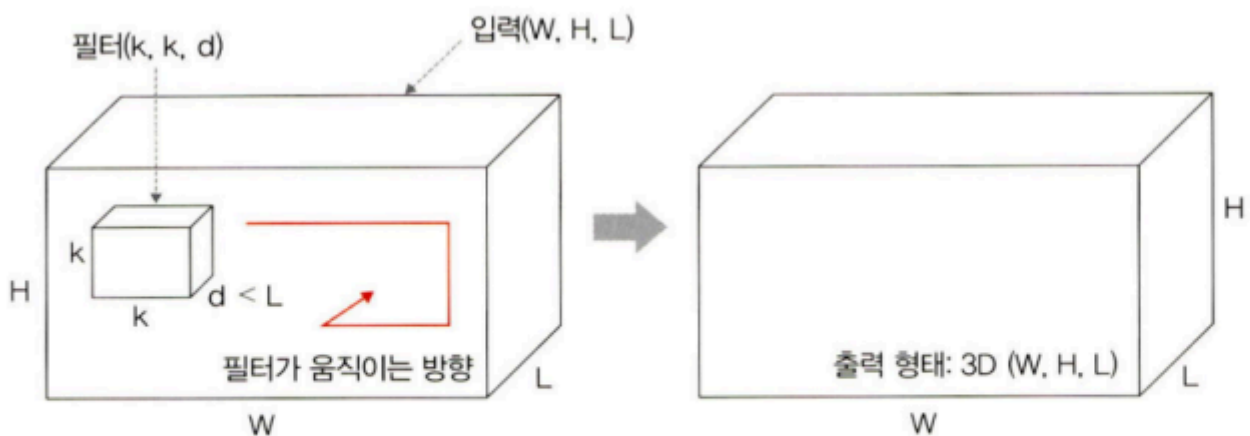
: 필터가 방향 세 개로 움직여 3D 형태로 출력, 입력(W, H, L)에 대해 필터 (k, k, d)를 적용했을 때 $d < L$ 을 유지해야 함

- 3D 입력을 갖는 2D 합성곱

: 필터에 대한 길이 L 이 입력 채널의 길이 L 과 같아야 하기 때문에 만들어지는 합성곱 형태

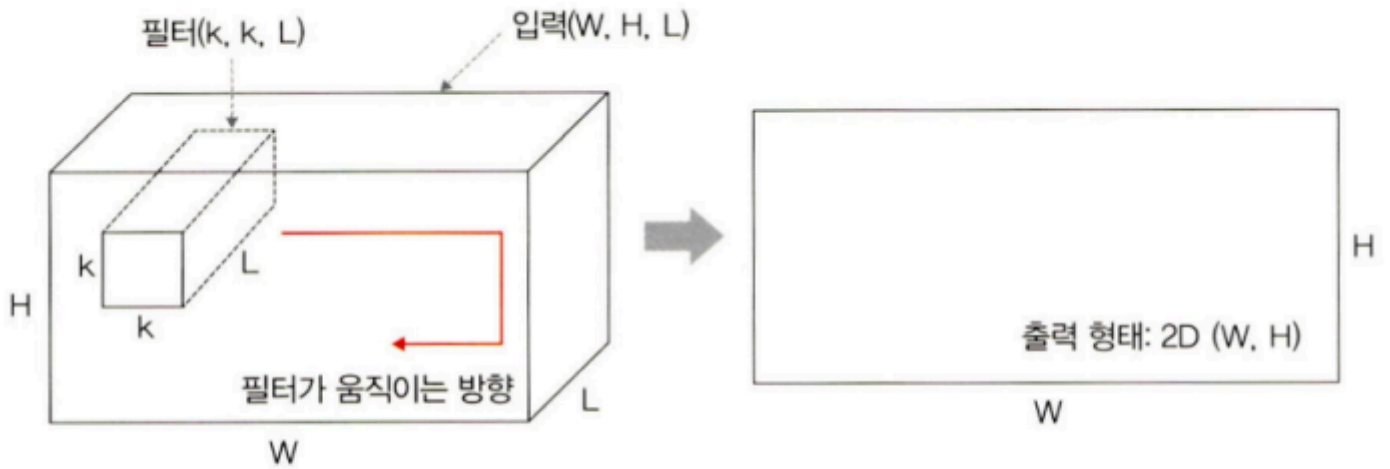
ex) LeNet-5, VGG

♥ 그림 5-21 3D 합성곱



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k \times d$ (높이 $\times$ 너비), d : 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)

▼ 그림 5-22 3D 입력을 갖는 2D 합성곱



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

3D 합성곱과 3D 입력을 갖는 2D 합성곱

- 1 x 1 합성곱

(W, H, L) 형태로 입력됨, 필터가 ($1, 1, L$)로 적용. 채널 수를 조정해서 연산량이 감소됨. (W, H)로 출력

ex) GoogLeNet

5.2 합성곱 신경망 맛보기

- fashion_mnist 데이터셋: 토치비전에 내장된 예제 데이터로, 작은 이미지들의 모음. 훈련 데이터는 넘파이 배열, 레이블 데이터는 0부터 9까지의 정수값을 갖는 배열.(옷 종류 클래스)
- GPU 사용

```
# 하나의 GPU 사용 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu") model = Net()
model.to(device) # 다수의 GPU 사용 device = torch.device("cuda" if torch.cuda.is_available() else "cpu") model
= Net() if torch.cuda.device_count() > 1: model = nn. DataParallel(net) model.to(device)
```

- fashion_mnist 데이터셋 내려받기

```
torchvision.datasets.FashionMNIST("위치 지정", download=True, transform=)
```

파라미터 설명

> download: True일 시 위치에 해당 데이터셋 있는지 확인 후 내려받음

> transform: 이미지를 텐서(0~1)로 변경

- 데이터로더 전달

: 내려받은 데이터를 메모리로 불러오기 위한 코드

`torch.utils.data.DataLoader(데이터를 불러올 데이터셋, batch_size=100)`

파라미터 설명

> batch_size: 데이터를 배치로 묶어줌

- 분류에 사용될 클래스 정의

`random.rand` vs `random.randn` vs `random.randint` `np.random.rand(8)` # 0~1 사이의 정규표준분포 난수를 1x8의 행렬로 출력 `np.random.randint(10)` # 0~10의 임의 숫자 출력 `np.random.randn(4,2)` # 평균이 0이고, 표준편차가 1인 가우시안 정규분포 난수를 4x2의 행렬로 출력

20개의 랜덤 이미지가 시각적으로 표현됨

심층 신경망 모델 생성

`nn`: 딥러닝 모델 구성에 필요한 모듈이 모여 있는 패키지

`nn.Linear()`: 선형 회귀 모델

`nn.Dropout(p)`: p만큼의 비율로 텐서값 0, 0이 되지 않는 값은 기존값에 $(1/(1-p))$ 만큼 곱해져 커짐

`forward()`: 순전파 연산 진행 함수

`.view()`: 넘파이의 `reshape`와 같은 역할

`F.relu()`: `forward()`에서 정의 / `nn.ReLU()`: `__init__()`에서 정의

▼ 표 5-1 `nn.xx`와 `nn.functional.xx`의 사용 방법 비교

구분	<code>nn.xx</code>	<code>nn.functional.xx</code>
형태	<code>nn.Conv2d</code> : 클래스 <code>nn.Module</code> 클래스를 상속받아 사용	<code>nn.functional.conv2d</code> : 함수 <code>def function (input)</code> 으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	<code>nn.Sequential</code> 내에 위치	<code>nn.Sequential</code> 에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

활성화 함수 정의 사용 방법 차이

```
class FashionDNN(nn.Module):  
    def __init__(self):  
        super(FashionDNN, self).__init__() # FashionDNN 클래스 상
```

```

속 받음 self.fc1 = nn.Linear(in_features=784, out_features=256) # in_features: 입력 크기, out_features: 출력 크
기 self.drop = nn.Dropout(0.25) self.fc2 = nn.Linear(in_features=256, out_features=128) self.fc3 =
nn.Linear(in_features=128, out_features=10) def forward(self, input_data): # 실제 데이터 연산 진행됨, 순전파
학습 진행 out = input_data.view(-1, 784) # 넘파이의 reshape와 같은 역할 out = F.relu(self.fc1(out)) # 활성화 함
수 정의 out = self.drop(out) out = F.relu(self.fc2(out)) out = self.fc3(out) return out

```

심층 신경망에서 필요한 파라미터 정의

경사 하강법은 Adam 사용(torch.optim.Adam(model.parameters(), lr=learning_rate));

심층 신경망을 이용한 모델 학습 및 성능 평가

```

images, labels = images.to(device), labels.to(device) # 모델과 데이터는 동일한 장치에 있어야 함,
model.to(device)와 같은 위치 train = Variable(images.view(100, 1, 28, 28)) # Autograd 패키지 사용, Variable()
사용해서 역전파를 위한 미분 값 자동으로 계산 classification accuracy = correct predictions / total predictions # 분
류 문제에 대한 정확도

```

합성곱 네트워크 생성

nn.Sequential: 계층을 차례로 쌓을 수 있도록 수식과 활성화 함수를 연결해 주는 역할, 하나의 컨테이너에 여러 개의 계층을 구현하는 방법

nn.Conv2d(): 합성곱층 추출 코드, in_channels=입력 채널 수, out_channels=출력 채널 수, kernel_size=커널 크기 (필터), padding=패딩 크기, 출력 크기 조정 위해 입력 데이터 주위에 0을 채움. 패딩 값이 클수록 출력 크기도 커짐.

nn.BatchNorm2d(): 각 배치 단위별로 데이터가 다양한 분포를 가져도 평균과 분산을 이용해 정규화하는 것

nn.MaxPool2d(): 이미지 크기를 축소시키는 용도, 풀링 계층 처리.

nn.Linear 사용해서 데이터 형태 1차원으로 변경(합성곱층->완전연결층)

```

class FashionCNN(nn.Module): def __init__(self): super(FashionCNN, self).__init__() self.layer1 =
nn.Sequential( nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(kernel_size=2, stride=2) ) self.layer2 = nn.Sequential(
nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3), nn.BatchNorm2d(64), nn.ReLU(), nn.
MaxPool2d(2) ) self.fc1 = nn.Linear(in_features=64*6*6, out_features=600) self.drop = nn.Dropout2d(0.25)
self.fc2 = nn.Linear(in_features=600, out_features=120) self.fc3 = nn.Linear(in_features=120,
out_features=10) def forward(self, x): out = self.layer1(x) out = self.layer2(out) out = out.view(out.size(0),
-1) # out.size(0) = 100을 의미 out = self.fc1(out) out = self.drop(out) out = self.fc2(out) out = self.fc3(out)
return out

```

Conv2d 계층에서의 출력 크기 구하는 공식

- 출력 크기 = $(W - F + 2P) / S + 1$
 - W : 입력 데이터의 크기(input_volume_size)
 - F : 커널 크기(kernel_size)
 - P : 패딩 크기(padding_size)
 - S : 스트라이드(strides)

예를 들어 첫 번째 Conv2d 계층은 다음과 같습니다.

MaxPool2d 계층에서의 출력 크기 구하는 공식

- 출력 크기 = IF / F
 - IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
 - F : 커널 크기(kernel_size)

완전연결층으로 보내기 위해 1차원 변경 시 출력 크기 구하는 공식

합성곱 네트워크를 위한 파라미터 정의, 모델 학습 및 성능 평가

딥러닝 신경망과 동일

=> 비교 결론: 심층 신경망과 비교하여 정확도가 약간 높음(86 vs 88)